NOTE You can rely on Spring Cloud Bus to broadcast configuration changes even when you use other options like Consul (with Spring Cloud Consul), Azure Key Vault (Spring Cloud Azure), AWS Parameter Store or AWS Secrets Manager (Spring Cloud AWS), or Google Cloud Secret Manager (Spring Cloud GCP). Unlike Spring Cloud Config, they don't have built-in push notification capabilities, so you need to trigger a configuration change or implement your monitor functionality manually.

14.1.3 Managing secrets with Spring Cloud Config

Managing secrets is a critical task for any software system, and it's dangerous when mistakes are made. So far, we have included passwords either in property files or environment variables, but they were unencrypted in both cases. One of the consequences of not encrypting them is that we can't version-control them safely. We would like to keep everything under version control and use Git repositories as the single sources of truth, which is one of the principles behind the GitOps strategy I'll cover in chapter 15.

The Spring Cloud Config project is well-equipped with features to handle configuration for cloud native applications, including secrets management. The main goal is to include secrets in the property files and put them under version control, which can only be done if they are encrypted.

Spring Cloud Config Server supports encryption and decryption and exposes two dedicated endpoints: `/encrypt` and `/decrypt`. Encryption can be based on a symmetric key or asymmetric key pair.

When using a symmetric key, Spring Cloud Config Server decrypts secrets locally and sends them decrypted to the client applications. In production, all communications between applications will happen over HTTPS, so the response sent from Config Service will be encrypted even if the configuration property is not, making this approach secure enough for real-world usage.